

עבודה מספר 1 ++C

הנחיות הגשה:

- יש להגיש את קבצי העבודה (8 קבצים סה"כ ללא assignment1.cpp) במערכת ה VPL ולוודא שאין שגיאות קומפילציה במערכת. **אין להגיש את הקובץ assignment1.cpp, קובץ זה נמצא במעטפת ההרצה של VPL.**
- ההגשה מתבצעת בזוגות (**שימו לב:** בעבודה זאת שני הסטודנטים יצטרכו להגיש את העבודה). בנוסף נדרש מכם להוסיף בהערות את השמות והתז של שני השותפים.
- יש להתאים את הפלטים של התוכנית שלכם לפלטים של קובץ ההרצה לדוגמא.
- הקפידו על הוראות ברורות וקריאות למשתמש לפני כל פעולת קלט.
- יש להקפיד של שמות המחלקות והשיטות כפי שמצוין בעבודה, הקפידו על אותיות גדולות וקטנות (אי התאמה תגרום לשגיאת קומפילציה בבדיקה האוטומטית).
- הקפידו על תכנות נכון, זהו חלק מן הציון:
 - שמות משתנים בעלי משמעות.
 - שימוש חוזר בקוד.
 - הקפדה על הזחות.
 - שימוש נרחב בהערות באנגלית בלבד. הקפידו לתעד כל שיטה וכל קטע קוד משמעותי.
- בראש כל קובץ יש להוסיף בהערה את הטקסט הבא:

```
*/Assignment C++: 1
Author: Israel Israeli, ID: 01234567
/*
```

- עליכם להחליף את הפרטים בפרטים שלכם (כמובן של שני השותפים לעבודה).
- כל שאלה יש להפנות לפורום המתאים לשאלות במודל.

תרגיל מס' 1

נושא התרגיל: מחלקה-פונקציה בונה, הורסת, public, private, העמסת אופרטורים.

הערה חשובה: רשימת המתודות היא חלקית בלבד.

היא מהווה את המינימום הנדרש. ייתכן ואף סביר להניח שתצטרכו להוסיף מתודות **נוספות** עבור המחלקות המתוארות. חישבו היטב מה באמת צריך בכל מחלקה. לכל המחלקות חובה להוסיף Constructor ו- Destructor דיפולטיביים (אפילו אם הוא ריק). על הבנאי הדיפולטיבי לאתחל את האובייקט בערכים הגיוניים. כמו כן עליכם להוסיף מתודות set ו- get **לפי הצורך**. בתרגיל זה אנו נבנה את המחלקות הבאות:

1. טיפוס נתונים מופשט - Abstract Data Type (ADT)

טיפוס נתונים מופשט (Abstract Data Type או ADT) הוא מודל מתמטי עבור קבוצה מסוימת של מבני נתונים בעלי התנהגות דומה. טיפוס נתונים מופשט מוגדר על ידי הפעולות שניתן לבצע עליו ועל ידי מגבלות שחלות על תוצאות הפעולות האלה (או העלות שלהן מבחינת סיבוכיות זמן ומקום). לדוגמה, מחסנית היא טיפוס נתונים מופשט הכולל פעולת Push שמוסיפה איבר, פעולת Pop שמסירה את האיבר האחרון שנוסף למחסנית, ופעולת Peek המאפשרת גישה למידע בראש המחסנית ללא הסרתו. בהקשר של ניתוח סיבוכיות של אלגוריתמים העושים שימוש במחסנית, ניתן להוסיף דרישות המקילות את ניתוח הסיבוכיות: זמן קבוע לכל פעולה, ללא תלות בכמות הנתונים במחסנית, ושטח זיכרון קבוע עבור כל איבר.

ניתן לקרוא עוד בקישור הבא: https://en.wikipedia.org/wiki/Abstract_data_type.

עליכם לממש את מחסנית דינמית (מחסנית שהגודל שלה משתנה בהתאם לצורך), לשם כך ממשו את המחלקה הבאה:

StackNode

Private members:

data – int next –

StackNode* **Stack**

Private members:

top – StackNode* Public

members:

Stack() – Initialize top to NULL.

push(int) - Inserts a new element at the top of the stack.

pop() - Removes the top element on the stack. If stack is empty, print "Stack is empty".

isEmpty() - Returns true if the stack is empty; otherwise, it returns false peek() -

Returns the top element present in the stack without modifying the stack.

In case where the stack is empty, return INT_MIN (need to include <climits>).

The return type is integer.

Operators:

- + - Concatenate two 'Stack' objects. i.e. $S_1 + S_2$ returns $\{S_1, S_2\}$.

For example: let $S_1 = \{1, 5\}$, $S_2 = \{8, 9\}$

Then, $S_1 + S_2 = \{1, 5, 8, 9\}$.

Cout << $S_1 + S_2$;

```
1. 1
2. 5
3. 8
4. 9
```

- + - Concatenate 'Stack' object with an integer. i.e. $S_1 + 3$ or $3 + S_1$ returns $\{S_1, 3\}$ or $\{3, S_1\}$ respectively.
- += - push an integer to the stack. i.e. $S += 4$ returns $\{4, S\}$.
- == - Compare two stacks and return true if they contain the same elements in the same order, and false otherwise.
- << - Insertion operator for easy printing.

הערה: ככלל אצבע, בכל מקום שאפשר לשים const עליכם לשים const. אתם נדרשים לחשוב לבד על חתימות הפונקציות. לדוגמא: הפונקציה peek() לא מבצעת שינוי באובייקט המפעיל ולכן היא יכולה להיות מוגדרת כ const. במידה והיא לא תוגדר כ const אזי מחסניות שהן קבועות לא יוכלו להפעיל אותה (ואין סיבה שלא נוכל לעשות זאת).

2. המחלקה MyQueue:

תור (queue) הוא מבנה נתונים מופשט המוגדר על ידי הפעולות הבאות:

הכנסה - (enqueue) הוספת אובייקט אחד חדש בסופו של התור

הוצאה - (dequeue) הוצאתו של האובייקט הנמצא בראש התור

בדיקה אם התור ריק (isEmpty) בדיקת ערך בראש התור (peek)

פעולות ההכנסה וההוצאה בתור מבוססות על העקרון נכנס ראשון - יוצא ראשון FIFO. עליכם לממש

את התור בעזרת שימוש בוקטור. וקטור הוא מערך דינאמי במובן שגודלו יכול להשתנות לאורך

התוכנית, אין חובה להקצות אותו בצורה דינאמית כמובן.

לקריאה נוספת על וקטורים:

<https://www.geeksforgeeks.org/vector-in-cpp-stl>

<http://www.cplusplus.com/reference/vector/vector>

private members:

vector<int> – contain the element inside the queue.

int maxQ – the capacity of the queue.

Public members:

MyQueue (int maxQ).

enqueue(int val) - insert a new element at the end of the queue. Return true if succeed and false otherwise.

dequeue() – remove the next element in the queue. Return true if succeed and false otherwise. isEmpty() – return true if the queue is empty and false otherwise.

peek() – returns the value of the first element in the queue (return int by value). If the Queue is empty, return -.1

3. מחלקת תפריט – Menu

מחלקה זו תנהל את המערכת. על מחלקה זו לבצע פעולות קלט/פלט מהמשתמש למעט שיטות שהוגדרו לכך במפורש במחלקות אחרות. יש להציג את התפריט בלולאה עד שהמשתמש יבחר לסיים אותה.
שיטות:

mainMenu - שיטה זו תדפיס את התפריט הבא:

המשתמש מקיש	שם הפעולה	הערות
1	כניסה לתפריט מחסנית	לאחר הקשה זו המערכת תפעיל מתודה של תפריט מחסנית
2	כניסה לתפריט מחסנית	לאחר הקשה זו המערכת תפעיל מתודה של תפריט מחסנית
3	יציאה מהתוכנית	לאחר הקשה זו המערכת תדפיס Goodbye! ותצא

stackMenu - שיטה זו תדפיס למסך תפריט להפעלת המחסנית.

המשתמש מקיש	שם הפעולה	הערות
1	הוספת איבר למחסנית	הקשה זו תוסיף איבר למחסנית בהתאם לקלט שמתקבל מהמשתמש
2	הוצאת איבר מהמחסנית	הקשה זו תשלוף איבר מהמחסנית
3	בדיקה אם המחסנית ריקה	הקשה זו תדפיס הודעה מתאימה בהתאם לתכולת המחסנית
4	הדפסת איברי המחסנית	הקשה זו תציג את איברי המחסנית לפי הסדר
5	יציאה מהתוכנית	יציאה מתפריט זה וחזרה לתפריט הראשי

queueMenu() - בעת הפעלת שיטה זו התוכנית תיתן למשתמש לבחור את גודל התור. לאחר מכן המשתמש יוכל לבצע פעולות על התור.

המשתמש מקיש	שם הפעולה	הערות
1	הדפסת התור	
2	הוספת איבר לתור	
3	הסרת איבר מהתור	
4	הדפסת האיבר הראשון בתור	
5	יציאה מהתוכנית	יציאה מתפריט זה וחזרה לתפריט הראשי

ה main -

מייצרת אובייקט מסוג תפריט ומריצה את המתודה "תפריט ראשי" וזהו.

הערות:

1. הקלטים יהיו מהטיפוסים החוקיים. ז"א בכל מקום נכניס את הטיפוס המתאים. אנחנו לא מתחייבים שהוא יהיה בטווח מסוים - אלא אם כן נאמר אחרת בשאלה עצמה.
2. אחרי כל הדפסה יש לבצע ירידת שורה.

יש להקפיד על תכנות נכון:

- a. כל הערכים שהם קבועים (מבחינה לוגית הם לא אמורים להשתנות), חייבים להיות מוגדרים כ: `const`, `define` או, `enum` בהתאם לצורך.
 - b. יש לרשום הערות בשפע, ובאנגלית בלבד (לכל מחלקה מה התפקיד שלה, התוכנית מה היא מבצעת, כל פעולה לא טריוויאלית להסביר, וכל 2~3 שורות קוד – הערה, כל מתודה מה היא עושה).
 - c. יש להקפיד על כימוס נכון – כל השדות ומתודות השירות ב- `private`, הממשק ב- `public`, חלוקה לקבצים.
 - d. יש להקפיד על הזחות, כיתוב נכון וקריא, ושמות משמעותיים.
 - e. יש לנסות ולייעל את הקוד והתוכנית ככל שניתן. הקפידו על `reuse` בקוד. נקודות ירדו על קוד כפול !
 - f. לפני בקשת קלט, יש להדפיס למשתמש הוראה (איזה קלט מבוקש).
 - g. להזכירכם: הנכם נדרשים לתכנת בשפת C++ אי-לכך, כל שימוש בפונקציות וספריות של שפת C אסורה.
3. עבור הצלחת הבדיקות עליכם לוודא שהדפסות זהות לקובץ ההרצה שקיבלתם. (לא יורדו נקודות על שוני ברווחים ושורות).
 4. בהצלחה!